



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/이미지처리

[이미지처리] 2. CNN모델링2

endingo 2025. 4. 8. 20:36

1. 데이터 셋 다운로드

```
train_dataset = datasets.FashionMNIST(root='data', train=True, download=True, transform=ToTensor())
test_dataset = datasets.FashionMNIST(root='data', train=False, download=True, transform=ToTensor())
```

2. 모델링

1) 모델선언

모델이 학습 할 수록 과적합이 일어나서
Droupout_rate = 0.5로 적용해보았다.

```
nn.Conv2d(1,32, kernel_size=3, stride=1, padding=1)
nn.MaxPool2d(kenel_size=2) 사
```

```
n_class=10
dropout_rate = 0.5
```



정훈이의 공부일지



```
nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),nn.ReLU(),nn.Dropout(p=dropout_rate), # 첫 번째 드롭아웃
nn.MaxPool2d(kernel_size=2),
nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),nn.ReLU(),nn.Dropout(p=dropout_rate), # 두 번째 드롭아웃
nn.MaxPool2d(kernel_size=2),
nn.Flatten(),
nn.Linear(64*7*7, 128),nn.ReLU(),nn.Dropout(p=dropout_rate), # 번째 드롭아웃
nn.Linear(128, n_class)
).to(device)
```

자 이렇게 쓰면 큰일나지는 않지만 옳은 방법은 아니다... 드롭아웃 선형 모델과 cnn모델에서 다르게 써야된다.

바로 Dropout2d(p=dropout_rate) 이런 형태로 써야된다.

- 수정된 모델 선언

```
n_class=10
dropout_rate = 0.5
```

```
model = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),nn.ReLU(),nn.Dropout2d(p=dropout_rate), # 첫 번째 드롭아웃
    nn.MaxPool2d(kernel_size=2),
    nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),nn.ReLU(),nn.Dropout2d(p=dropout_rate), # 첫 번째 드롭아웃
    nn.MaxPool2d(kernel_size=2),
    nn.Flatten(),
    nn.Linear(64*7*7, 128),nn.ReLU(),nn.Dropout(p=dropout_rate), # 첫 번째 드롭아웃
    nn.Linear(128, n_class)
).to(device)
```

동일한 조건에서의 dropout_rate 조정:

잘못된 코드 최고점수: 91.2

옳은 코드 최고점수: 93.7

2) 목적함수, 옵티마이저

```
loss_fn =nn.CrossEntropyLoss()
```



```

epochs = 10
tr_loss_list, val_loss_list, val_acc_list = [], [], []

# early stopping을 위한 변수 설정
patience = 5
best_loss = float('inf') # 초기값을 무한대로 설정
counter = 0

# 모델 저장 경로
PATH = './model.pt'

for t in range(epochs):
    tr_loss = train(train_dataloader, model, loss_fn, optimizer, device)
    val_loss, pred = evaluate(x_val, y_val, model, loss_fn, device)

    # accuracy 측정
    pred = nn.functional.softmax(pred, dim=1)
    pred = np.argmax(pred.cpu().numpy(), axis = 1)
    acc = accuracy_score(y_val.numpy(), pred)

    # 리스트에 추가
    tr_loss_list.append(tr_loss) # train - CrossEntropy
    val_loss_list.append(val_loss) # val - CrossEntropy
    val_acc_list.append(acc) # val - Accuracy

    print(f"Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}, val acc : {acc:.4f}")

    # early stopping 확인 -----
    if val_loss > best_loss:
        counter += 1
        print(f"--> early stopping status, best_loss : {best_loss:.4f}")
    else:
        best_loss = val_loss
        counter = 0
        torch.save(model, PATH) # <- best model 저장하기

    # 조기 종료 조건 확인
    if counter >= patience:
        print("Early Stopping!")
        break
    # -----

```



정훈이의 공부일지

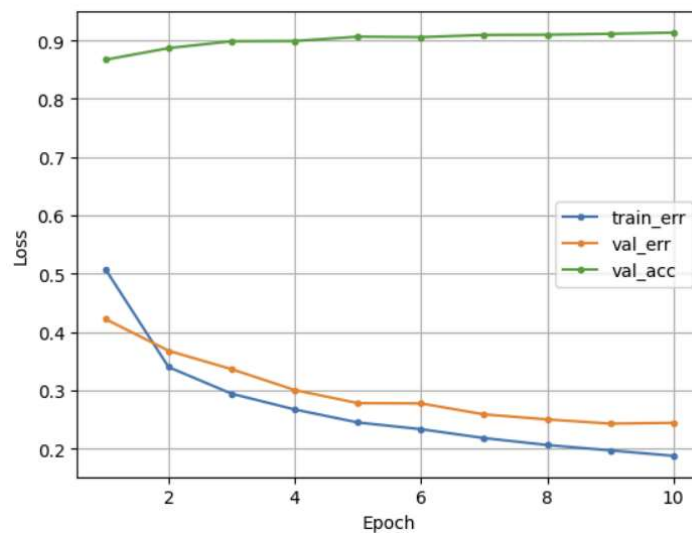


```
Epoch 7, train loss : 0.2182, val loss : 0.2588, val acc : 0.9094
Epoch 8, train loss : 0.2063, val loss : 0.2502, val acc : 0.9098
Epoch 9, train loss : 0.1971, val loss : 0.2430, val acc : 0.9114
Epoch 10, train loss : 0.1878, val loss : 0.2443, val acc : 0.9134
-> early stopping status, best_loss : 0.243009
```

과적합 문제 해결된 것을 볼 수 있다.

4) 모델 평가

dl_learning_curve(tr_loss_list, val_loss_list, val_acc_list)



train err, val_err 점차 감소

val_acc 점점 증가

아마 epoch 증가시키면 더 증가 할 것 같다.

- 평가하고 확률로 바꾼후 가장 높은 확률 값을 선

```
_, pred = evaluate(x_test, y_test, model, loss_fn, device)
pred[:5]
```

```
pred = nn.functional.softmax(pred, dim=1)
pred = np.argmax(pred.cpu().numpy(), axis = 1)
#pred = np.argmax(pred.numpy(), axis = 1)
pred
```

5) 혼동행렬 확인

```
cm = confusion_matrix(y_test.numpy(), pred)
cm
```

confusion matrix 시각화

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels = train_dataset.classes)
```



정훈이의 공부일지



```
print(accuracy_score(y_test.numpy(), pred))
print('-'*100)
print(classification_report(y_test.numpy(), pred))
```

```
0.9192
```

	precision	recall	f1-score	support
0	0.85	0.89	0.87	493
1	1.00	0.97	0.98	519
2	0.93	0.85	0.89	479
3	0.91	0.96	0.93	500
4	0.83	0.90	0.86	479
5	0.99	0.98	0.99	515
6	0.78	0.72	0.75	518
7	0.95	0.96	0.96	500
8	0.98	1.00	0.99	474
9	0.97	0.96	0.96	523
accuracy			0.92	5000
macro avg	0.92	0.92	0.92	5000
weighted avg	0.92	0.92	0.92	5000

이처럼 이미지처리에는 cnn을 이용하면 linear를 이용할 때 보다 더 성능이 높게 나옴을 알 수 있다.

공감

구독하기

[Data science](#) > [이미지처리](#) 카테고리의 다른 글

[\[이미지처리\] 1. CNN모델링1](#) (0)

정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

구독하기



정훈이의 공부일지

이름

비밀번호

로그인 댓글만 허용한 블로그입니다

등록

공지사항	최근에 올라온 글	최근에 달린 댓글	Total
	[이미지처리] 2. CNN모델링2	덕분에 좋은 정보 얻어 갑니다 ! ...	200
	[자바 문제풀이] 1. IQ KING		Today 5
	[이미지처리] 1. CNN모델링1		Yesterday 16
	[딥러닝] 9. 클래스 선언 방식		

링크

TAG

more

git

github

« 2025/04 »

일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

글 보관함

2025/04 (21)

2025/03 (7)

Blog is powered by Tistory / Designed by Tistory



이정훈 개발자

AI와 백엔드, 데이터 분석을 좋아하는 개발자입니다.
다양한 문제 해결과 창의적인 개발을 추구합니다.

 이메일 보내기